

算法设计与分析讲义

Lecture notes on Algorithm Design and Analysis (LADA)

黄宇

南京大学计算机科学与技术学院

<https://github.com/alg-nju/lada>

2026 年 9 月 16 日

使用说明

- 讲义的一章大致对应于一次授课。每一章的开头会有一个内容概要，列出此次授课所有的知识要点。
- 讲义的内容在持续写作中。目前优先写进讲义的内容，是当前授课所用的课本和胶片中，讲得不足需要补充和调整的部分。也就是说，讲义尚未写出来的内容，在课本和胶片中可以找到相应的详细描述。

前言

前言。

[2]。

目录

第一部分 准备知识	5
第一章 计算模型 (L1)	7
1.1 抽象的计算机：计算模型的引入	7
1.1.1 算法的直观概念	7
1.1.2 Pointer模型：RAM模型的对象化改造	8
第二章 函数的渐近增长率 (L2)	10
第三章 分治递归求解 (L3)	11
3.1 Master定理	11
第四章 算法视角下的数学知识 (T1)	14
 第二部分 从蛮力到分治	 16
第五章 快速排序 (L4)	18
5.1 快速排序	19
5.1.1 原地地划分的实现	19
第六章 堆排序 (L5)	21
第七章 归并排序 (L6)	22
7.1 归并排序的设计	22
7.1.1 归并过程的空间复杂度	22
7.2 归并排序的分析	23
7.2.1 最坏情况下的归并代价	23
7.3 基于比较的排序算法的下界	24
第八章 从排序到分治 (T2)	25
8.1 归并分治	25
8.1.1 Closest pair问题的补充解释	25

目录	4
第九章 线性时间选择 (L7, L8)	27
9.1 分组大小背后的微妙权衡	27
第十章 对数时间查找 (L9)	29
第十一章 分治的平衡性 (T3)	30
 第三部分 从图遍历到图优化	 31
第十二章 图遍历	33
12.1 图遍历的抽象框架	33

第一部分

准备知识

准备知识部分。

第一章 计算模型 (L1)

授课提纲

计算模型

- 计算的基本概念
- 抽象算法的概念
- RAM模型的引入
- Pointer模型的拓展¹

算法设计

- 正确性的标准：问题的规约 (specification)
- 证明正确性的手段：数学归纳法
- 示例：EUCLID算法的正确性证明

算法分析

- 抽象的算法分析：一个计数 (counting) 问题
- 时间复杂度分析：最坏情况 vs. 平均情况
- 空间复杂度分析
- 一个简单的示例：数组扫描算法

1.1 抽象的计算机：计算模型的引入

1.1.1 算法的直观概念

我们首先不严格地讨论算法的直观概念。

算法是“the step-by-step procedures that computers use to solve problems”²。

¹在开始讲图算法时，正式引入。

²这一说法源自于Quanta Magazine的文章《Computer scientists establish the best way to traverse a graph》，作者Ben Brubaker，链接：<https://www.quantamagazine.org/computer-scientists-establish-the-best-way-to-traverse-a-graph-20241025/>。

我们可以借用“类比”(analog)来解释算法相关的概念³。假设我们了解一台计算机及其上运行的程序,那么我们的“算法”,就是“计算机程序在数学世界中的类比”(mathematical analog of a computer program)。我们描述算法所用的“伪码”,就是“计算机编程语言在数学世界中类比”(mathematical analog of a programming language)。这里所说的伪码是指面向算法描述的,结构化的自然语言,可以是中文、英文等。我们的“计算模型”,就是“现实世界中的计算机在数学世界中的类比”(mathematical analog of a computer)。

表 1.1: 概念的对比

数学世界的概念	计算机世界的概念
算法	程序
伪码语言	编程语言
计算模型	计算机

1.1.2 Pointer模型：RAM模型的对象化改造

在算法设计与分析的过程中,我们经常需要用到动态分配的数据对象,例如各种链表、以邻接链表表示的图等。基础的RAM模型并不直接支持“寻址”的概念,表达动态数据结构的时候,会非常地不方便。为了便于使用各种常用的动态数据结构,我们引入Pointer模型的概念⁴。

在Pointer模型中,我们可以动态分配、修改、释放数据对象(object)。每个数据对象,包含若干个(但是不超过一个常数的上界)字段(field),每个字段可以是一份存储单元中的数据,也可以是一个指针(pointer),指针指向另一个数据对象——这也是Pointer模型名字的由来。我们还特别约定指针可以取特殊值NULL,表示其不指向任何数据对象。

我们可以在基本的RAM模型中,实现上述指针模型。简单而言,我们可以用存储单元的序号(index)来表示对象的位置,进而实现指针的概念。在实际使用中,我们不详细讨论实现细节,只是假设我们的RAM模型具有动态分配数据对象,寻址一个数据对象并进行各种操作的能力。

章节注释

算法(algorithm)一词的由来

花拉子米(780年代-850年代⁵),波斯数学家。他的《代数学》是在约830年写成的一部数学著作,这是第一本解决一次方程及一元二次方程的系统著作,他因而被称为代数的创造者。代数的英文单词“algebra”就出自于他的著作中对于一元二次方程的一种解法。花拉子米在825年写成的《印度数字算术》(On the Calculation with Hindu Numerals)一书对于印度-阿拉伯数

³这里参考了Erik Demaine教授在课堂上的讲授。课程信息: MIT open courseware, Introduction to algorithms (6.006, fall 2011)。链接: <https://courses.csail.mit.edu/6.006/fall11/notes.shtml>。最后访问日期: 2026.8.5。
⁴同样参考了Erik Demaine教授在MIT 6.006算法课程中的相关讲解,详见§1.1.1的脚注。
⁵大约是中国历史上安史之乱之后,唐朝中晚期时代。

字系统在中东及欧洲的传播尤其重要。《印度数字算术》被翻译成拉丁语 “Algoritmi de numero Indorum”，而 “算法” (algorithm) 一词则来自于花拉子米名字的拉丁文音译。

是“模型源自于实践”，还是“模型指导了实践”

在我们的授课过程中，对于算法知识的学习和编程实践之间的关系，我们是有预设的要求的。我们的授课，假设大家有基本的编程知识，并且有过一定量的基础编程实践。在此基础上，希望我们的算法设计和分析课程能够帮助大家更好地认识到程序的正确运作、高效运作背后的原理。基于这一“设定”，这次课所讨论的作为抽象算法设计与分析之基础的计算模型，更像是从具体的实践中归纳出来的。

深入思考上面的设定，大家有可能会有这样的困惑。讨论算法的源头，一般都会提到欧几里得⁶算法，它大概形成于中国的战国时期。中国的三国时期的数学家刘徽所提出的割圆法，也是一种优美的算法。为什么在一个电都没有的年代，却可以有算法。另一个更著名的算法问题是希尔伯特第十问题⁷，它要求寻找判定丢番图方程是否有解的算法。但究竟什么是算法呢？在希尔伯特提出问题之时却并不存在一个明确定义。

最终图灵机的出现，对什么是计算，什么是算法，给出了精确的定义。图灵机首先是一个纯粹的数学抽象，其诞生动机来自数理逻辑内部的问题，而非当时物理计算技术的直接反映。如果非要说有什么“得益于”的地方，那也许是当时的打字机、磁带等日常机械给了图灵关于“读写头+移动纸带”这一直观隐喻。倒是物理的计算机的制造，得益于图灵机相关理论概念的发展。图灵机先定义了通用计算的数学边界，后来的冯·诺依曼架构、存储程序计算机，在概念上受到了图灵抽象的间接影响。图灵本人二战后参与建造的 ACE (Automatic Computing Engine) 明确借鉴了他在1936年提出的图灵机相关的理论。

所以回到我们的这次课程。从计算机科学技术的历史来看，我们更应该理解为抽象的模型指导了具体的实践。但是在我们今天学习这门本科生基础课程的时候，我们已经站在了巨人的肩膀上。此时，我们的授课方法是，以充分的实践为诱因和助力，更好得学习和理解算法的知识。

⁶Euclid, 公元前325年——公元前265年

⁷关于希尔伯特第十问题的材料有多种，这里主要参考了维基百科和卢昌海老师的《Hilbert第十问题漫谈》

第二章 函数的渐近增长率 (L2)

授课提纲

基本概念

- 渐近增长率概念的由来
- O 的定义：基于 $\epsilon - N$ 语言的定义 vs. 基于求极限的定义
- Ω 和 Θ 的定义：由 O 的定义派生而来
- o 和 ω 的定义：强调档次上的差距
- 经典算法代价函数的分档： $\log n, n, n \log n, n^2, n^3(n^k), 2^n, n!$

Brute-Force (BF) 范式

- BF范式的基本概念：以排序、选择、查找为例
- BF范式的方法学意义：认识问题的难度，寻找改进的机会；以算法分析为“导航”
- 经典例题：数组左右交换问题；最大和连续子串问题

章节注释

微积分严格化的历史

$\epsilon - \delta$ 语言的背后，是微积分严格化的艰苦历程。

牛顿和莱布尼茨虽然把微积分系统化，但是它还是不够严谨。可是当微积分被成功地用来解决许多问题，却使得十八世纪的数学家偏向其应用，而甚少致力于其严谨性。一些数学家，包括科林·麦克劳林，试图证明使用无穷小是可靠的做法，但直到150多年之后才得以成功。

奥古斯丁·路易·柯西和卡尔·魏尔斯特拉斯的工作，实现了对无穷小的记号的回避。微分和积分的基础终于被打下了。在柯西的著作中，可以看到一系列的基础进路尝试，包括通过无穷小来对连续进行定义，和在微分定义中一个不太精确的函数极限原型。而在魏尔斯特拉斯的著述中，对极限概念作了形式化，回避了无穷小的使用。继魏尔斯特拉斯之后，微积分就常以极限作为基础，而非无穷小了。

第三章 分治递归求解 (L3)

授课提纲

分治策略的基本概念

- 递归，作为一种思维模式；解递归方程，作为一种算法分析手段
- 从简单递归到分治策略
- 分治的典型模式（难分易合、易分难合）与样例

解递归的基本技术

- 忽略取整，笨展开
- Guess & Prove：一种定制版的数学归纳法

分治递归求解

- 笨展开+Recursion Tree
- Master定理，三种情况的由来，等比数列：公比大于1、等于1、小于1
- Case 3的进一步解释

3.1 Master定理

定理 3.1 (Master定理). 令 $a \geq 1$ 和 $b > 1$ 为常数， $f(n)$ 为一定义于非负整数上的函数。 $T(n)$ 为定义于非负整数上的递归函数：

$$T(n) = aT\left(\frac{n}{b}\right) + f(n) \quad (3.1)$$

递归式中的 $\frac{n}{b}$ 指的是 $\lfloor \frac{n}{b} \rfloor$ 或者 $\lceil \frac{n}{b} \rceil$ 。 $T(n)$ 的渐近增长率为：

1. 如果存在某个常数 $\epsilon > 0$ ，使得 $f(n) = O(n^{\log_b a - \epsilon})$ ，则 $T(n) = \Theta(n^{\log_b a})$ 。
2. 如果 $f(n) = \Theta(n^{\log_b a})$ ，则 $T(n) = \Theta(n^{\log_b a} \log n)$ 。
3. 如果存在常数 $\epsilon > 0$ ，使得 $f(n) = \Omega(n^{\log_b a + \epsilon})$ ，且存在某个常数 $c < 1$ ，使得对所有充分大的 n ， $af(\frac{n}{b}) \leq cf(n)$ ，则 $T(n) = \Theta(f(n))$ 。

证明. 根据图3.1, 我们有:

$$T(n) = \underbrace{\Theta(n^{\log_b a})}_{\text{叶节点层代价}} + \underbrace{g(n)}_{\text{所有非叶结点层代价}} \quad (3.2)$$

$$g(n) = \sum_{j=0}^{\lfloor \log_b n \rfloor} a^j f\left(\frac{n}{b^j}\right) \quad (3.3)$$

对于 $g(n)$ 的求和形式, 注意当 j 取0时, 我们得到 $f(n)$ 。此外, 求和的项数必须是整数。递归树的层数记为 $\lfloor \log_b n \rfloor + 1$, 则非叶子节点有 $\lfloor \log_b n \rfloor$ 层, 即 $g(n)$ 的求和形式有 $\lfloor \log_b n \rfloor$ 项。

第一种情况. 已知 $f(n) = O(n^{\log_b a - \epsilon})$, 所以 $f(\frac{n}{b^j}) = O((\frac{n}{b^j})^{\log_b a - \epsilon})$ 。据此, 我们有 $g(n)$ 满足:

$$\begin{aligned} g(n) &= \sum_{j=0}^{\lfloor \log_b n \rfloor} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \epsilon} \\ &= n^{\log_b a - \epsilon} \sum_{j=0}^{\lfloor \log_b n \rfloor} \left(\frac{ab^\epsilon}{b^{\log_b a}}\right)^j \\ &= n^{\log_b a - \epsilon} \sum_{j=0}^{\lfloor \log_b n \rfloor} (b^\epsilon)^j \\ &= n^{\log_b a - \epsilon} \left(\frac{b^{\epsilon \log_b n} - 1}{b^\epsilon - 1}\right) \\ &= n^{\log_b a - \epsilon} \left(\frac{n^{\epsilon - 1}}{b^\epsilon - 1}\right) \end{aligned}$$

上面的推导过程中, 为了表示的方便, 我们省去了 $g(n)$ 表达式中的渐进增长率符号 $O(\cdot)$ 。因为 b 和 ϵ 为常数, 所以我们有 $g(n) = O(n^{\log_b a - \epsilon} n^\epsilon) = O(n^{\log_b a})$ 。所以根据公式3.2, 我们证明了 $T(n) = \Theta(n^{\log_b a})$ 。

第二种情况. 已知 $f(n) = \Theta(n^{\log_b a})$, 则 $f(\frac{n}{b^j}) = \Theta((\frac{n}{b^j})^{\log_b a})$ 。据此, 我们有 $g(n)$ 满足:

$$\begin{aligned} g(n) &= \sum_{j=0}^{\lfloor \log_b n \rfloor} a^j \left(\frac{n}{b^j}\right)^{\log_b a} \\ &= n^{\log_b a} \sum_{j=0}^{\lfloor \log_b n \rfloor} \left(\frac{a}{b^{\log_b a}}\right)^j \\ &= n^{\log_b a} \sum_{j=0}^{\lfloor \log_b n \rfloor} 1 \\ &= n^{\log_b a} \log_b n \end{aligned}$$

上面的推导过程中, 为了表示的方便, 我们同样省去了 $g(n)$ 表达式中的渐进增长率符号 $\Theta(\cdot)$ 。根据公式3.2, 我们证明了 $T(n) = \Theta(n^{\log_b a} \log n)$ 。

第三种情况. 根据 $T(n)$ 的原始表达式 (公式3.1), 我们知道 $T(n) = \Omega(f(n))$ 。所以, 为了证明 $T(n) = \Theta(f(n))$, 我们的任务只剩下证明 $T(n) = O(f(n))$ 。根据第三种情况的第一个条件: 存在常数 $\epsilon > 0$, 使得 $f(n) = \Omega(n^{\log_b a + \epsilon})$ 。根据 $T(n)$ 的递归树展开表达式 (公式3.2), 我

们有 $T(n)$ 的第一项 $n^{\log_b a} = O(f(n))$ 。此时我们的任务只剩下证明 $T(n)$ 的第二项 $g(n) = O(f(n))$ 。而这一结论，完全是第三种情况的第二个条件，给“硬性规定出来”的。

对于第三种情况，已知 $af(\frac{n}{b}) \leq cf(n)$ ，所以 $a^2f(\frac{n}{b^2}) = a \cdot af(\frac{n}{b^2}) \leq a \cdot cf(\frac{n}{b}) \leq c^2f(n)$ 。上述展开可以作用任意 j 次： $a^j f(\frac{n}{b^j}) \leq c^j f(n)$ 。所以根据公式3.3，我们有 $g(n)$ 满足：

$$\begin{aligned} g(n) &= \sum_{j=0}^{\lfloor \log_b n \rfloor} a^j f\left(\frac{n}{b^j}\right) \\ &\leq \sum_{j=0}^{\lfloor \log_b n \rfloor} c^j f(n) \\ &\leq f(n) \sum_{j=0}^{\infty} c^j \quad (c < 1) \\ &= O(f(n)) \end{aligned}$$

由此，我们证明了 $T(n) = \Theta(f(n))$ ¹。

□

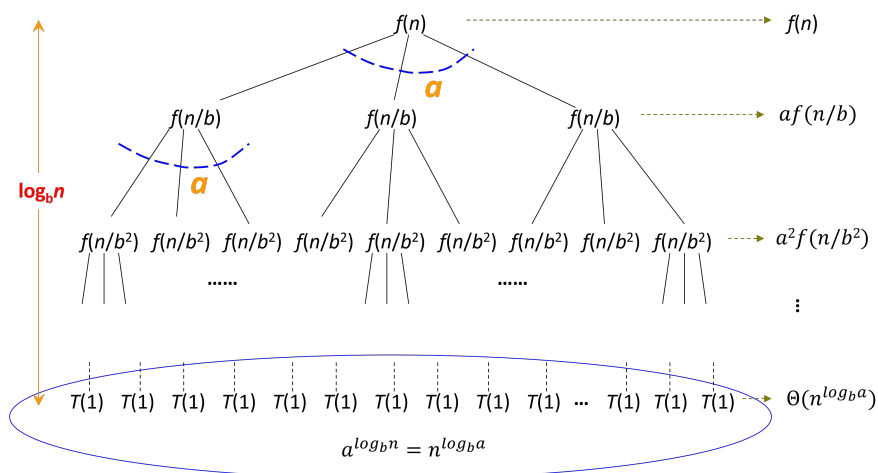


图 3.1: $T(n)$ 的递归树展开

¹证明过程参考了《算法导论》的第三版和第四版的相关章节。图示基于陈道蓄老师的授课胶片修改。

第四章 算法视角下的数学知识 (T1)

If you find the mathematics difficult, skip those early parts. Later sections will be understandable provided you are willing to forgo the deep insights mathematics gives into the weaknesses of our current beliefs. General results are always stated in words, so the content will still be there but in a slightly diluted form.

— Richard W. Hamming [3]

授课提纲

I - 公理化视角

从直觉开始构造

- 数学归纳法的基础：良序原理
- 自然数集合的公理化构造
- 数域的公理化构造： $N \Rightarrow Z \Rightarrow Q \Rightarrow R \Rightarrow C$

数学归纳法证明算法正确性

- 归纳法辨析：“所有马同色”谬误
- 归纳法应用：EUCLID, BUBBLESORT, PRIM

认识和理解不变式 (invariant) 的概念

- 归纳法中的“变”与“不变”
- 更大范围地应用不变式的概念

II - 数学的概念，算法的视角

基本数学概念

- 函数： $f(n)$
- 取整： $\lfloor x \rfloor, \lceil x \rceil$

- 对数: $\log_a b$
- 阶乘: $n!$

基本级数

- 多项式级数: 算术级数, 平方和, 任意 k 次方和 (的渐近增长率)
- 几何级数: 一般形式, 2和 $\frac{1}{2}$ 的特例, 渐进增长率的速判
- 算术几何级数: “错项相减”法
- 调和级数
- 斐波那契数列

面向平均情况时间复杂度分析的期望值求解

- 指标随机变量
- 期望的线性性 (linearity)
- 示例: 排序算法平均情况时间复杂度的数学描述

III - BF范式: 分析驱动设计改进

BF范式 (paradigm)

- BF是一个“相对性”的概念
- 基本模式: 分析发现不足 $\Rightarrow$ 大O度量不足 $\Rightarrow$ 不足推动改进

分析驱动改进

- “名人”问题; 不变式: 一次比较, 确定淘汰一个
- 频繁项问题: $k = 2$ 的特例; 不变式: $|f| > |\neg f|$

第二部分

从蛮力到分治

从蛮力到分治。

第五章 快速排序 (L4)

授课提纲

I - BF排序

最朴素的SELECTIONSORT和BUBBLESORT

- $W(n)$: $\binom{n}{2}$ 的必然性
- $A(n) = W(n)$

INSERTIONSORT的改进

- 局部有序性的利用
- $W(n)$: 不变
- $A(n)$: 定量地降了一半

蛮力排序的不足

- inversion的概念
- inversion的数量: 最多、平均
- 蛮力算法: 消除inversion速度的不足

II - 快速排序

快速排序的设计原理

- 如何一次比较, 消除更多的inversion
- 分治策略的典型应用: 难分易合

快速排序的时间复杂度

- $W(n)$: 最不平衡的划分
- $A(n)$: 列递归, 解递归
 - smart guess, guess & prove

– 直接求解：错项相减

- $A(n)$ ：指标随机变量、变量的linearity¹

III - 空间复杂度

空间复杂度的分档

- $O(1)$ 的含义：原地 (in-place) 排序
- $O(n)$ 的含义

蛮力排序、堆排序²

- 显然的 $O(1)$ 空间

快速排序

- $O(n)$ 空间的朴素实现
- $O(1)$ 空间的改进：四个部分、一个不变式

归并排序：多次的反转³

- 只能 $O(n)$
- 可以 $O(1)$
- 还是 $O(n)$

5.1 快速排序

5.1.1 原地划分的实现

在做PARTITION的时候，开辟一个 $O(n)$ 的辅助空间，划分很容易实现。一个自然的想法是，能否将划分改为原地 (in-place) 的。

原地划分的做法是，用两个指针 i 和 j ，将元素划分为四个部分，如图5.1所示。划分算法维护的不变式是 i 和 j 界定了小于部分和大于部分。

算法当前待处理的元素是 $A[j]$ 。其他情况简单，唯一需要说明的是当 $A[j]$ 纳入一个小于pivot的元素时，不变式被违反。

此时的做法很巧妙：让 i 指针右移一位，它也违反了不变式。让 i 、 j 位置的元素互换，则两个违反都消失，整个数组恢复保持不变式。

¹将在tutorial（第八章）中专题讲解。

²将在相应课程（第六章）中针对性讲解。

³将在相应课程（第七章）中针对性讲解。

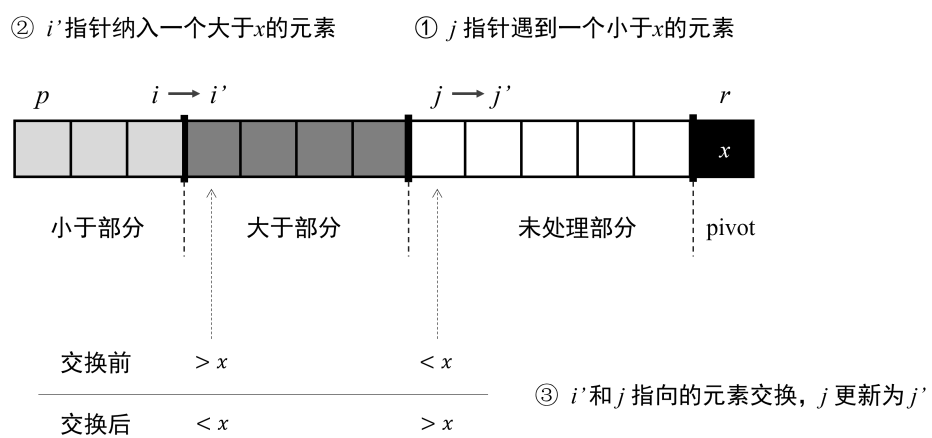


图 5.1: 原地的元素划分

第六章 堆排序 (L5)

授课提纲

堆的定义

- 二叉树的相关定义：递归的思维方式与定义手段
- 堆的定义：两个维度的解耦

堆算法

- 堆排序
- 堆的修复
- 堆的构建
- 堆结构的数组表示

第七章 归并排序 (L6)

授课提纲

I - 归并排序

MERGESORT的设计辨析

- 基本设计与分析
- 合并的最坏情况时间复杂度的问题 (§7.2.1)

整体性地考虑空间复杂度问题

- 从蛮力排序和堆排序，到快速排序，到归并排序
- 从 $O(n)$ 到 $O(1)$ ，再到 $O(n)$

II - 比较排序的下界

下界问题

- 概念辨析：最大的最小值；最小的最大值
- 下界的定义：最坏情况、平均情况

一个算法族的抽象建模

- 比较排序 $\Rightarrow$ 决策树，对应表 (§7.3)

比较排序的下界

- 最坏情况：二叉树的 h 和 L 的关系
- 平均情况：越平衡、EPL越小

7.1 归并排序的设计

7.1.1 归并过程的空间复杂度

随机存储（数组） vs. 顺序存储（链表）

数组支持随机访问（random access），即访问任意 $A[i]$ 都是 $O(1)$ 时间。但是数组是一段连续的存储空间，必须预先分配指定的大小。数组中插入元素时，需要涉及多个元素的挪动。

反之，链表只支持顺序访问（sequential access），访问链表中第 k 个元素的代价为 $O(k)$ 时间。但是链表支持更灵活的调整，插入、删除元素都是 $O(1)$ 时间。链表可以在使用时，即时分配。

$O(n)$ 的空间代价

对照分析MERGE和PARTITION算法，我们发现由于被MERGE的数组是有序的，所以他们无法随意交换元素位置，因而无法像PARTITION那样，简单地改成原地算法。这与§5.1.1的设计形成鲜明对比。

一种将空间代价降为 $O(1)$ 的“改进”

此时，一个看似合理的改进是：原地MERGE，数组被分为左右两半，分别记为 A 和 B 。当 B 头部的元素 b_1 更小时，我们可以用大约 $\frac{n}{2}$ 次SWAP，将 b_1 向左挪到合适的位置。通过多次的SWAP，我们实现了原地的MERGE。由于我们是比较的排序，上述的SWAP操作是完全“免费”的，所以改进后的MERGE算法，时间复杂度不变，空间复杂度变为 $O(1)$ 。

这一分析结论过程的问题，在于忽视了我们对于critical operation的要求。comparison之所以能够成为critical operation，一个必要的前提是，其他non-comparison的辅助操作的个数，必须跟comparison操作的个数满足线性关系。由此条件，我们才可以（在big-oh的意义下）仅计数 comparison 操作。

上述做法，1次comparison，要辅以 $O(n)$ 次SWAP操作，那它已经不再是comparison-based sorting，我们的分析就无效了。如果坚持这样的“改进”，那我们的建模和分析则应该同时统计 COMPARE 和 SWAP 操作的代价。分析结果表明这样的改进就不再有效了。

真的改进，却不是最广泛使用

对于链表，归并是可以在 $O(1)$ 空间完成的。一个比较著名的例子是Linux kernel中对于linked list的归并排序 [1]。

在实际使用中，归并排序算法，主要面向顺序存储上的数据排序。在随机存储上，一般快速排序、堆排序具有更好的性能。虽然上述 $O(1)$ 空间的改进可以实现，但是实际采用的，更多还是需要 $O(n)$ 空间的简单实现 [5]。

7.2 归并排序的分析

7.2.1 最坏情况下的归并代价

归并的最坏情况所对应的输入

假设我们将原始数组分成左右两部分，分别为 $A[1..\frac{n}{2}]$ 和 $B[1..\frac{n}{2}]$ ，现在我们要将他们合并到空数组 C 中。

对于MERGE过程而言，最坏情况输入是 A 和 B 的元素在大小关系下是交叉的，即：

$$b_1 < a_1 < b_2 < a_2 < \cdots < b_{n/2} < a_{n/2} \quad (7.1)$$

反过来说，MERGE的最好情况是，有尽量多的元素无需比较，直接完成合并，即： $a_1 < a_2 < \cdots < a_{n/2} < b_1 < b_2 < \cdots < b_{n/2}$ 。此时，所有A的元素通过比较，都挪到C中。当B中的元素面对空的A数组时，无需任何比较，我们直接将整个B挪到C中。

一种似乎也是最坏的输入

此时，一个潜在的疑问是，公式（7.1）的要求是充分的，但是它否有必要。只要我保证A和B都一直不空（直至最后才空），是否就可以？

假设A和B的元素满足 $a_1 < a_2 < \cdots < a_{n/2-1} < b_1 < b_2 < \cdots < b_{n/2} < a_{n/2}$ 。这一构造的原理是：A数组在比较中，差点就空了，只剩一个元素 $a_{n/2}$ 特别大，不是也能保证最坏情况么？因为B的元素还是只能通过比较，逐个挪到C中。

下面的例子说明，公式（7.1）的要求是必要的，因为面对有序的数组，我们可以玩出一些特别的“花样”。

这样的输入做不到最坏

此时， $a_{n/2}$ 这一个元素，面对一个有序的数组 $B[1..n/2]$ ，我们可以用折半查找找到 $a_{n/2}$ 应该出现的位置，然后无需额外比较，直接把B和 $a_{n/2}$ 都挪到C中。

此时，基于折半查找，实际的比较次数大约是 $\frac{n}{2} + \log \frac{n}{2}$ 。所以这一情况，由于我们可以采用折半查找，所以它并未将MERGE逼迫到最坏情况。

7.3 基于比较的排序算法的下界

分析比较排序算法的下界，基本的技术手段是，将比较排序算法的运作，和决策树（decision tree）对应起来。这一对应关系，如表7.1所示。

表 7.1: 决策树与比较排序算法的对应关系

Decision Tree	Algorithm
internal node	binary decision (comparison)
leaf	found answer
root-to-leaf path	algorithm execution
path length	running time
height of the tree	worst-case running time

第八章 从排序到分治 (T2)

授课提纲

分治

- 分治

8.1 归并分治

归并排序使用分治策略的模式，具有通用性，应用场合不仅限于排序：

- 最大和子串。
- Inversion计数。
- Closest pair (cf. §8.1.1)。
- Maxima。
- Frequent items。
- Integer multiplication。
- Matrix multiplication。

8.1.1 Closest pair问题的补充解释

我们可以保守地估计，当前检查的点 q_i 上下所有跟它距离可能更近的点，也就是当前检查的点 q_i 在一个 4×4 的网格中，最多需要比较它周边15个点 [4, §5.4]。

大家对于这一更保守的估计反而有点疑惑，就是它为什么考虑了很多显然可以约减的情况。针对这一情况，我们改用下面更细致一点的估计。 q_i 只需要考虑它下面的点，连 q_i 自己，一共8个格子，所以 q_i 至多需要考虑7个点。

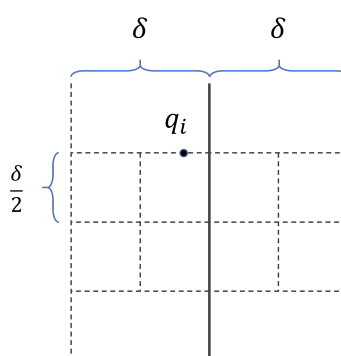


图 8.1: Closest pair问题

第九章 线性时间选择 (L7, L8)

9.1 分组大小背后的微妙权衡

我们将期望线性时间的选择算法记为SELELIN，将最坏线性时间的算法记为SELWLIN。

在SELWLIN算法中，有一个“神奇”的数字“5”。想要深入理解SELWLIN算法，一个自然的疑问就是，为什么元素的分组大小应该是5？或者说，其他的分组大小——例如分组大小为3、4、7、9等——行不行？

看起来分组的大小有无无数种选项，但是我们可以经过简单的分析，将关注点聚焦到少数代表性选项，这可以帮助我们有效理解分组大小“5”背后的原理。首先不难理解，偶数的分组大小可以被排除，因为每组有奇数个元素时，不仅中位数的定义更加简洁，而且更有利于保证最终划分的平衡性。其次，分组大小的选择，以5为基准，有两个方向，一个方向是将分组变得更“细”，此时只有分组大小为3这一个选择。另一个方向是将分组变得更“粗”，这有无穷多种选择，但是我们可以对分组大小7做一个深入剖析，进而理解分组大小选择5的原理。所以我们下面对代表性的分组大小3和7，进行深入分析。

分组大小为3的情况

如果我们将SELWLIN算法中分组的大小定为3，而算法的其他设计保持不变，此时算法的运作更接近于SELELIN中所有元素一字排开的情况。考虑到SELELIN算法的最坏情况时间复杂度 $O(n^2)$ ，我们有理由担心算法的最坏情况时间复杂度是否能保持在 $O(n)$ 的水平。基于上面的考虑，我们试图去证明算法时间复杂度的一个下界，若能证明算法的时间复杂度严格超过了 $O(n)$ ，则可以表明分组大小3不可行。

根据算法的划分方式，我们可以确定比 m^* 小的元素，至多有：

$$2 \times \left\lceil \frac{1}{2} \left\lceil \frac{n}{3} \right\rceil \right\rceil \leq \frac{n}{3} + 3$$

上述不等式左边的原理很简单，大致估算下来，所有分组中，有一半的小组，每个小组有2个元素，确定比 m^* 小。由于此时我们只需要估算一个上界，所以我们在除以2或者3的时候，都取上取整，在计算小组数量时，也未扣除 m^* 所在的小组。与SEL5算法的分析类似，此时最坏情况下，我们需要对至少 $n - (\frac{n}{3} + 3) = \frac{2}{3}n - 3$ 个元素进行递归。所以我们有算法的最坏情况时间复杂度满足：

$$W(n) \geq W\left(\left\lceil \frac{n}{3} \right\rceil\right) + W\left(\frac{2}{3}n - 3\right) + \Omega(n)$$

$W(n)$ 递归不等式右边的第一项，是对所有小组的中位数，选择 m^* 的代价；第二项是最坏情况下，对最多的元素进行递归选择的代价；第三项是元素划分的代价。通过“guess and prove”的方式我们不难证明 $W(n) = \Omega(n \log n)$ ，具体证明留作习题。

上述分析表明，分组大小若是设为3，则 $W(n)$ 的渐进增长率严格高于 $O(n)$ ，所以分组大小为3的选项在设计SELWLIN算法时，被合理抛弃。

分组大小为7的情况

以分组大小5为基准，更大的取值代表着更精细的分组和更好地平衡性控制（当 n 充分大的时候），所以我们有理由相信算法的最坏情况代价仍然能保持 $O(n)$ 的水平，而不会出现分组大小为3时， $W(n) = \Omega(n \log n)$ 的情况。基于这一合理猜测，我们来严格证明 $W(n) = O(n)$ 。

考虑7个元素一小组的划分方式，则确定比 m^* 小的元素至少有：

$$4 \times \left(\left\lceil \frac{1}{2} \left\lceil \frac{n}{7} \right\rceil \right\rceil - 2 \right) \geq \frac{2}{7}n - 8$$

上述不等式左边的原理是，我们大致有一半约 $\frac{n}{7} \times \frac{1}{2}$ 个小组，每个小组贡献4个元素（每组上方有3个元素，再加上组内的中位数）确定比 m^* 小。在处理取整的时候，我们为了确定得到一个下界，所以保守地去掉了 m^* 所在的小组，也去掉了最后一个可能并不完整的小组（由于 n 可能不是7的倍数，所以最后一小组的元素可能不足7个），所以共去掉2个小组。

所以在最坏情况下，算法要对不超过 $\frac{5}{7}n + 8$ 的元素进行递归，所以算法的最坏情况时间复杂度满足：

$$W(n) \leq W\left(\left\lceil \frac{n}{7} \right\rceil\right) + W\left(\frac{5}{7}n + 8\right) + O(n)$$

同样通过“guess and prove”的方式，我们不难证明 $W(n) = O(n)$ ，具体证明留作习题。

上述证明表明了我们的猜测是准确的，将分组变得更细（从5个一组变成7个一组）并不会使算法的时间复杂度变坏。但是在 O 的意义下，分7个一组也并不会对算法的时间复杂度带来本质改进，而只会让算法的具体实现变得更复杂。这是因为选择问题显然具有 $\Omega(n)$ 的下界，而 $O(n)$ 的SELWLIN算法已经是最优的。

以3和7为代表性的情况，我们对SELWLIN算法中分组大小的影响进行了定量分析。上述分析表明，选择5个元素分为一小组是一个合适的选择。

第十章 对数时间查找 (L9)

授课提纲

查找

- 查找

第十一章 分治的平衡性 (T3)

授课提纲

分治

- 分治

第三部分

从图遍历到图优化

图遍历。

图优化：贪心、动态规划。

第十二章 图遍历

12.1 图遍历的抽象框架

WHATEVERFIRSTSEARCH (WFS), 算法1。

Algorithm 1 WHATEVERFIRSTSEARCH(s)

```
1: put  $s$  into the bag  $B$ 
2: while  $B$  is not empty do
3:   take  $v$  from  $B$ 
4:   if  $v$  is unmarked then
5:     mark  $v$ 
6:     for all edge  $vw$  do
7:       put  $w$  into  $B$ 
```

WFS的外层调用算法 (wrapper), 算法2。

Algorithm 2 WFSWRAPPER(G)

```
1: for all vertices  $v$  do unmark  $v$ 
2: for all vertices  $v$  do
3:   if  $v$  is unmarked then WFS( $v$ )
```

参考文献

- [1] list_sort.c in the Linux kernel. https://github.com/torvalds/linux/blob/master/lib/list_sort.c. Last accessed: Sep. 2026.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Ctein. *Introduction to Algorithms (Fourth edition)*. the MIT Press, 2022.
- [3] Richard W. Hamming. *The Art of Doing Science and Engineering: Learning to Learn*. Taylor & Francis, 2005.
- [4] Jon Kleinberg and Eva Tardos. *Algorithm Design*. Prentice Hall, 2006.
- [5] Wikipedia. Merge sort. https://en.wikipedia.org/wiki/Merge_sort. Last accessed: Sep. 2026.